Project Report: MNIST Digit Recognition & Overfitting Analysis

1. Objective

The goal of this project is to implement an image digit recognition system using the MNIST dataset, deliberately induce model overfitting using a high-capacity architecture, and subsequently mitigate that overfitting using training-based regularization techniques—all without altering the underlying model architecture.

2. Dataset & Data Splitting

The project utilizes the standard **MNIST dataset**, which consists of 70,000 grayscale images of handwritten digits (0-9). The data was rigorously split to ensure the model could be evaluated properly:

- **Training Set:** 50,000 images (used to actively train the model).
- **Validation Set:** 10,000 images (used to monitor overfitting during training).
- **Test Set:** 10,000 images (used for the final, unbiased evaluation).

A fixed random seed was used during the splitting process to ensure identical datasets across all experiments.

3. Model Architectures

To demonstrate the full spectrum of overfitting, two architectures were created:

A. Normal Baseline (NormalMLP)

A standard, reasonably-sized Multi-Layer Perceptron (MLP) consisting of:

- **Input Layer:** 784 pixels.
- **Hidden Layer:** 128 neurons (ReLU).
- **Regularization:** Added a Dropout layer (nn.Dropout(0.2)) to keep the baseline well-behaved and minimize the training-validation gap.
- **Output Layer:** 10 neurons.

B. High-Capacity Model (LargeMLP)

To successfully guarantee severe overfitting, an oversized model was constructed:

1. **Input Layer:** 784 pixels.
2. **Hidden Layer 1:** 1,024 neurons (ReLU).
3. **Hidden Layer 2:** 1,024 neurons (ReLU).
4. **Hidden Layer 3:** 512 neurons (ReLU).
5. **Output Layer:** 10 neurons.

This architecture contains roughly **1.8 million parameters**, granting it the "memory capacity" to easily memorize the training images if left unconstrained.

4. Phase 0: Normal Model Baseline

Before demonstrating severe overfitting, the NormalMLP (incorporating 20% Dropout) was trained for 15 epochs to establish a realistic, generalized baseline.
Results:

- **Training Accuracy:** 98.65% (Loss: 0.0400) I **Validation Accuracy:** 97.56% (Loss: 0.0881)
- **Test Accuracy:** 97.78%
- **Conclusion:** By using a reasonably-sized network combined with light dropout, the model generalizes well. The gap between Training Loss (0.0400) and Validation Loss (0.0881) is narrow, demonstrating a highly stable and well-regularized baseline.

5. Phase 1: Inducing Overfitting

During Phase 1, the LargeMLP model was trained for 15 epochs using the Adam optimizer without any regularization (no dropout, no weight decay, no data augmentation).

Results:

- The **Training Accuracy** reached near-perfect levels (~99.50%), and the Training Loss dropped to 0.0175.
- However, the **Validation Loss** diverged, rising back up to 0.1080 (Validation Accuracy: 97.71%).
- **Conclusion:** The massive 1.8M parameter capacity allowed the network to memorize the specific pixel details of the training set. The divergence between the decreasing training loss and increasing validation loss is the classic indicator of overfitting.

6. Phase 2: Mitigating Overfitting (Regularization)

The challenge in Phase 2 was to overcome the overfitting problem without modifying the LargeMLP class (meaning structural changes like adding Dropout or Batch Normalization layers were strictly prohibited).

To achieve this, two training-based regularization techniques were implemented:

1. **Data Augmentation:** Random rotations (up to 15 degrees) and random spatial translations (up to 10% shifts) were applied to the training images on the fly. Because the model never saw the exact same image twice, it was physically prevented from memorizing pixel layouts and was forced to learn the general shape of the digits.
2. **Weight Decay (L2 Regularization):** A mathematical penalty (weight_decay=1e-3) was added to the Adam optimizer. This penalty forced the optimizer to keep the neural network's weights as small as possible, effectively restricting the model's complexity and forcing it to find simpler, more generalized rules.

Results:

- While the **Training Accuracy** was lower compared to Phase 1 (dropping to ~96.14% and training loss to 0.1240), the **Validation Loss** dropped significantly to 0.0741 (Validation Accuracy: 97.56%).
- The **Test Accuracy** reached 98.14%, outperforming both the baseline and the overfitted model.
- **Conclusion:** The model successfully generalized to unseen data. By constraining the model's learning process through L2 regularization and data augmentation, we prevented memorization and built a highly robust digit recognizer.

7. Theoretical Justification

A. Bias-Variance Tradeoff & Model Capacity

In machine learning, the generalization error of a model is composed of bias, variance, and irreducible noise.

- **Bias** represents the error introduced by approximating a complex real-world problem with a simplified model.
- **Variance** represents the sensitivity of the model to the specific fluctuations in the training set.

By increasing the network's parameters from ~100k (NormalMLP) to ~1.8M (LargeMLP), we dramatically increased the model's capacity (hypothesis space size).

- In **Phase 1**, this high capacity allowed the model to achieve near-zero training loss (extremely low bias) by memorizing noise and training-specific fluctuations. However, this caused high variance, as evidenced by the validation loss (0.1080) diverging from the training loss (0.0175).

B. Weight Decay (L2 Regularization)

L2 regularization modifies the objective function by adding a penalty term proportional to the sum of the squared weights:
$\tilde{L}(\theta; X, y) = L(\theta; X, y) + \frac{\lambda}{2} |\mathbf{w}|^2_2$
Where λ is the weight decay coefficient (1e-3 in Phase 2).

- **Mathematical Effect:** Taking the gradient of this regularized loss yields weight updates of the form: $\mathbf{w} \leftarrow (1 - \eta\lambda)\mathbf{w} - \eta \nabla_{\mathbf{w}} L$.
- **Theoretical Justification:** This forces the weights to decay exponentially towards zero unless sustained by the data gradient. Keeping weights small restricts the effective complexity of the network (acting as a soft constraint on the parameter space), preventing the network from fitting high-frequency noise and smoothing the learned decision boundary.

C. Data Augmentation

Data augmentation is a form of *explicit regularization* that generates synthetic training samples by applying label-preserving transformations (random rotations and translations).

- **Theoretical Justification:**

    1. **Support Expansion:** It expands the empirical training distribution to better cover the true data manifold.
    2. **Invariance Learning:** It injects prior knowledge of affine invariances directly into the training loop, forcing the model to learn representations invariant to minor translations and rotations rather than relying on static pixel locations.

D. Dropout (used in Phase 0 Baseline)

Dropout randomly sets a fraction p (here, $p=0.2$) of the activation units to zero during each training forward pass.

- **Theoretical Justification:** Dropout can be viewed as training an ensemble of 2^d sub-networks with shared weights (where d is the number of hidden units). At test time, these sub-networks are averaged. Additionally, it prevents co-adaptation of features, meaning no single neuron can rely on the presence of another specific neuron to make a prediction, forcing the network to learn robust, redundant representations.

8. Summary & Metric Comparison

This project successfully highlights the dangers of using unconstrained, high-capacity neural networks on simple datasets. Furthermore, it demonstrates that architectural changes are not strictly necessary to fix overfitting; training-level techniques such as Data Augmentation and Weight Decay are highly effective tools for forcing a model to generalize.

Below is the summary of the final epoch results:

Metric	Phase 0 (Normal Baseline)	Phase 1 (Overfitting)	Phase 2 (Regularized)
Training Accuracy	98.65%	99.50%	96.14%
Training Loss	0.0400	0.0175	0.1240
Validation Accuracy	97.56%	97.71%	97.56%
Validation Loss	0.0881	0.1080	0.0741
Test Accuracy	97.78%	98.01%	**98.14%**